

SVEUČILIŠTE JOSIPA JURJA STROSSMAYERA U OSIJEKU

FAKULTET PRIMIJENJENE MATEMATIKE I INFORMATIKE

SEMINARSKI RAD

BAZA PODATAKA ZA RITMIČKU VIDEOIGRU

MODERNI SUSTAVI BAZA PODATAKA

Tibor Milković

Osijek, 2025.

Sadržaj

Uvod.....	1
MEV i Relacijski model.....	2
Kreiranje Tablica.....	5
Unos Podataka.....	7
Jednostavni upiti.....	8
Upiti nad više tablica.....	10
Upiti s agregirajućim funkcijama.....	13
Podupiti / ugniježdeni upiti / skupovne operacije.....	16
Zadane vrijednosti.....	18
Uvjeti.....	19
Komentari.....	20
Dodavanje indeksa.....	20
Procedure.....	21
Okidači.....	24
Zaključak.....	27

1. Uvod

Ritmičke videoigre, žanr koji zahtijeva brze reflekse i osjećaj za glazbu, u pozadini skriva ogromne baze podataka koje pamte svaki pogodak i promašaj igrača. Budući da sam i sam upoznat sa svijetom ovih igara, odlučio sam se za ovu temu kako bih istražio kako takav sustav funkcionira.

Ovaj rad prikazuje model baze podataka za jednu takvu igru, od osnovne strukture za praćenje korisnika i rezultata, pa sve do pravila koja igru čine dinamičnom i poštenom za sve igrače. Iako je model pojednostavljen, cilj mu je prikazati logiku i osnovne principe na kojima se temelje sustavi ovakvog tipa, implementirane u PostgreSQL sustavu za upravljanje bazama podataka.

2. MEV i Relacijski model

U MEV modelu definirani su osnovni entiteti i njihovi odnosi.

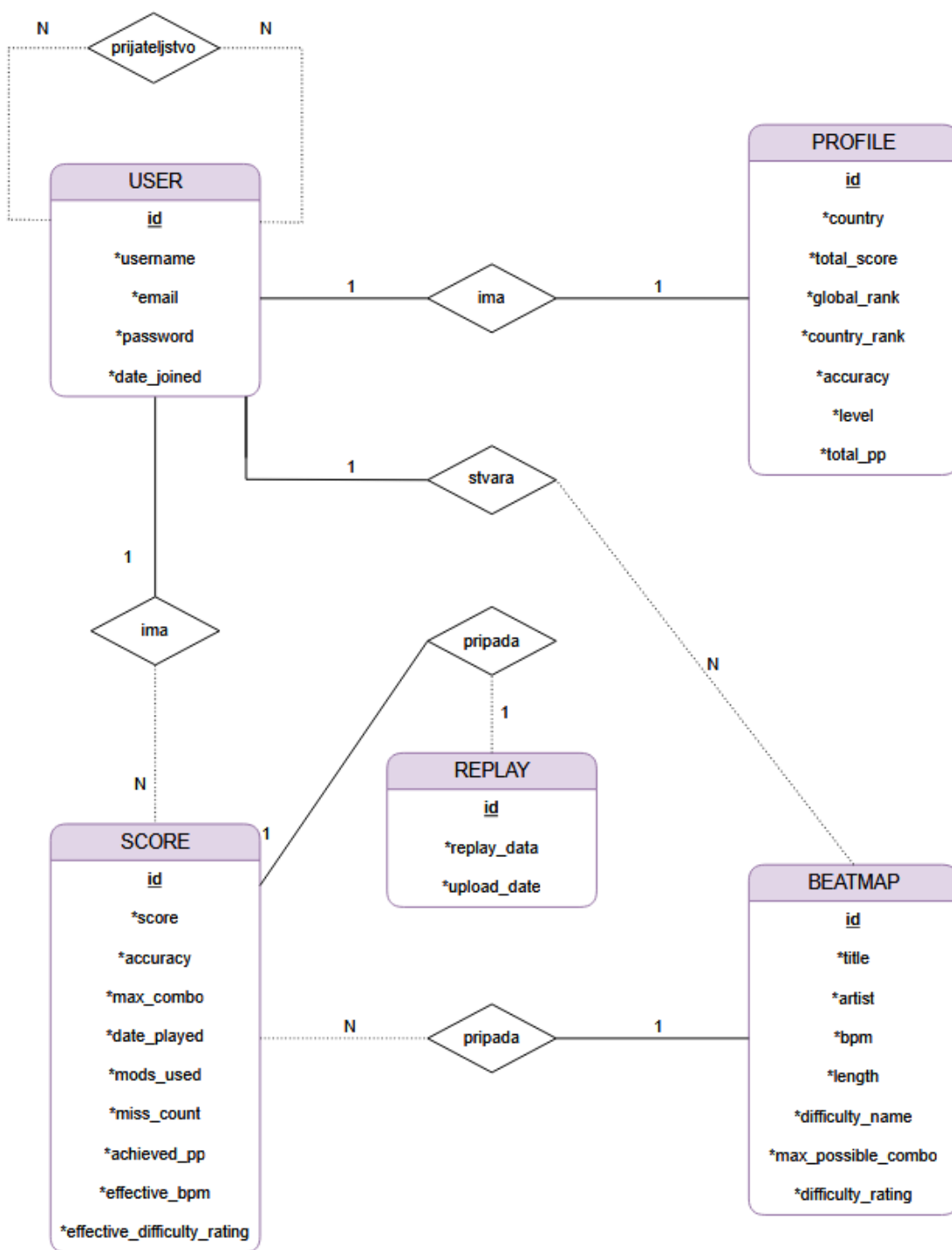
Definicija entiteta:

1. **USER:** Središnji entitet koji predstavlja svakog registriranog igrača. Sadrži osnovne podatke za prijavu i identifikaciju, poput korisničkog imena, emaila i lozinke.
2. **PROFILE:** Proširenje USER entiteta koje pohranjuje kumulativne statistike igrača. To uključuje ukupan broj bodova (total_score), globalni i državni rang, prosječnu preciznost (accuracy) i level. Svaki korisnik ima točno jedan profil (veza 1:1).
3. **BEATMAP:** Predstavlja pojedinu "mapu" ili "pjesmu" koju igrač može igrati. Sadrži sve relevantne podatke o pjesmi, kao što su naziv, izvođač, BPM (Beats Per Minute) i ocjena težine. Svaku mapu stvara jedan korisnik (kreator).
4. **SCORE:** Predstavlja zapis o jednom ostvarenom rezultatu na određenoj mapi. Ovo je najčešće pristupljeni entitet u bazi, odnosno u njega se konstantno upisuju novi podaci. On bilježi specifične podatke o performansama u toj partiji: ostvareni bodovi, preciznost, maksimalni kombo, broj promašaja (miss_count) i osvojene "performance points" (achieved_pp).
5. **REPLAY:** Opcionalni entitet koji je vezan uz točno jedan SCORE. Sadrži binarne podatke koji omogućuju ponovno gledanje (replay) odigrane partije.

Veze i kardinalnost:

- USER – PROFILE (1:1) - svaki korisnik ima točno jedan profil. Profil ne može postojati bez korisnika.
- USER – SCORE (1:N) – jedan korisnik može imati mnogo rezultata (scores), ali svaki rezultat pripada točno jednom korisniku.
- USER – BEATMAP(1:N) – jedan korisnik (kreator) može stvoriti više mapa, a svaka mapa ima jednog kreatora.
- BEATMAP – SCORE(1:N) – Na jednoj mapi može biti postignuto više rezultata, ali svaki rezultat je postignut na točno jednoj mapi.
- SCORE – REPLAY(1:1) – Svaki rezultat može imati najviše jedan replay koji služi za ponovno gledanje već odigranog.

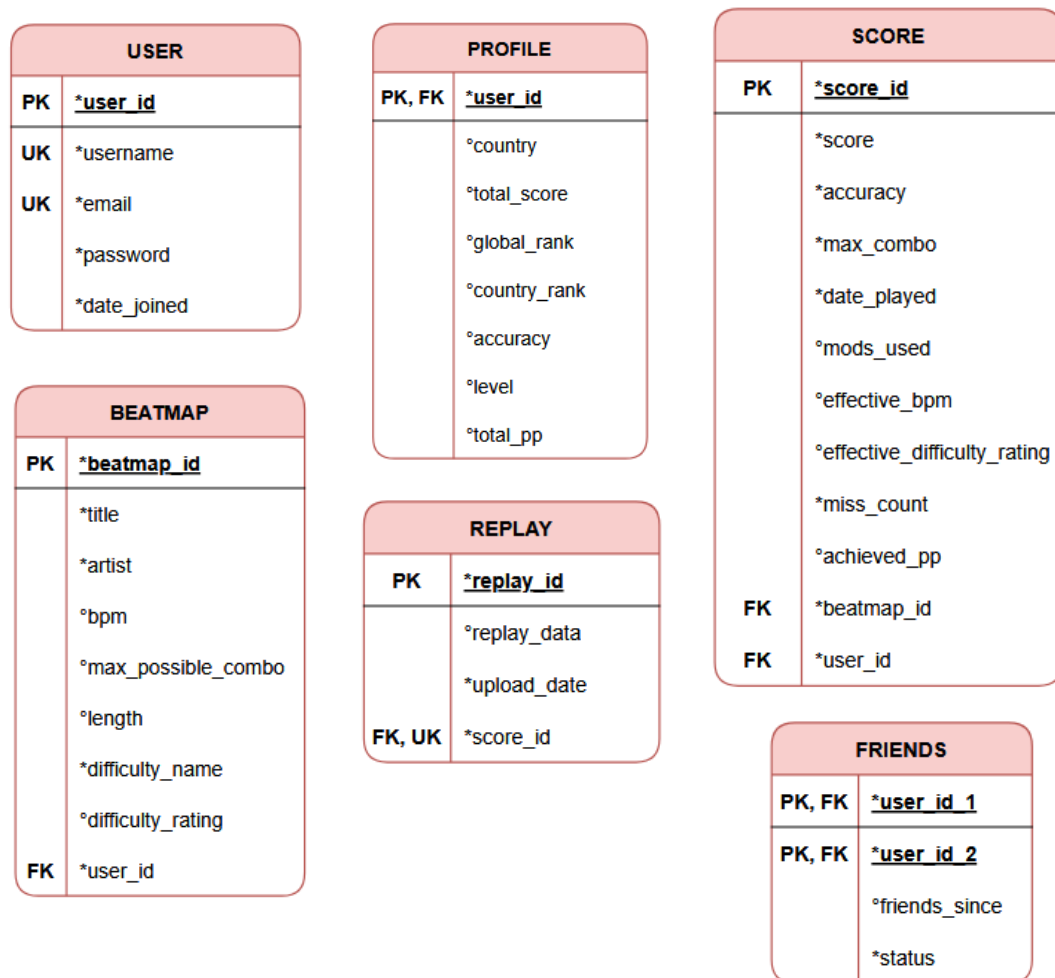
2.1 MEV model



2.2 Relacijski model

ER model je preslikan u relacijski model, gdje su entiteti postali tablice, a atributi stupci. Veze su implementirane pomoću primarnih i stranih ključeva.

- **Rješavanje N:N veze:** Veza prijateljstvo (N:N) riješena je uvođenjem asocijativne tablice friends. Ova tablica sadrži strane ključeve user_id_1 i user_id_2 koji referenciraju tablicu user, te dodatne attribute status i friends_since koji opisuju samu vezu.
- **Strani ključevi:** U relacijskom modelu jasno su definirani strani ključevi (FK) koji osiguravaju referencijalni integritet, pazeći da se, primjerice, ne može unijeti rezultat za korisnika koji ne postoji.
- **Nužnost atributa:** Precizno je definirano koji atributi moraju imati vrijednost (oznaka *), a koji su opcionalni (oznaka °), što je ključno za logiku igre (npr. friends_since je opcionalan jer nema vrijednost dok zahtjev nije prihvaćen).



3. Kreiranje Tablica

Na temelju relacijskog modela kreirano je šest tablica u PostgreSQL bazi podataka. Svaka tablica predstavlja jedan od ključnih entiteta igre. Ispod se nalazi nekoliko fotografija kreiranja najrelevantnijih tablica.

Brisanje tablica:

```
DROP TABLE IF EXISTS "user", profile, beatmap, score, replay, friends CASCADE;
```

Kreiranje tablica:

```
CREATE TABLE "user" (  
    user_id        SERIAL          PRIMARY KEY,  
    username       VARCHAR(50)     NOT NULL UNIQUE,  
    email          VARCHAR(100)    NOT NULL UNIQUE,  
    password       VARCHAR(255)    NOT NULL,  
    date_joined    TIMESTAMP       NOT NULL  
);
```

NAPOMENA: s obzirom da je **user** keyword u postgreSQLu, koristim "user" radi čitljivosti.

```
CREATE TABLE profile (  
    user_id        INTEGER          PRIMARY KEY,  
    country        VARCHAR(50),  
    total_score    BIGINT,  
    global_rank    INTEGER,  
    country_rank   INTEGER,  
    accuracy       NUMERIC(5,2),  
    level          SMALLINT,  
    total_pp       INTEGER,  
    CONSTRAINT fk_profile_user FOREIGN KEY (user_id) REFERENCES "user"(user_id)  
);
```

```

CREATE TABLE beatmap (
    beatmap_id      SERIAL          PRIMARY KEY,
    title           VARCHAR(200)    NOT NULL,
    artist          VARCHAR(200)    NOT NULL,
    max_possible_combo INTEGER,
    bpm            INTEGER,
    length          INTEGER,        -- izrazeno u sekundama
    difficulty_name VARCHAR(100)    NOT NULL,
    difficulty_rating NUMERIC(4,2),
    user_id        INTEGER          NOT NULL, -- user_id kreatora mape
    CONSTRAINT fk_beatmap_creator FOREIGN KEY (user_id) REFERENCES user(user_id)
);

```

```

CREATE TABLE score (
    score_id      SERIAL          PRIMARY KEY,
    score         BIGINT          NOT NULL,
    accuracy      NUMERIC(5,2)    NOT NULL,
    max_combo     INTEGER          NOT NULL,
    miss_count    INTEGER          NOT NULL,
    effective_bpm INTEGER,
    effective_difficulty_rating NUMERIC(4,2),
    date_played   TIMESTAMP        NOT NULL,
    mods_used     VARCHAR(200),
    achieved_pp   INTEGER,
    user_id       INTEGER          NOT NULL,
    beatmap_id    INTEGER          NOT NULL,
    CONSTRAINT fk_score_user FOREIGN KEY (user_id) REFERENCES user(user_id),
    CONSTRAINT fk_score_beatmap FOREIGN KEY (beatmap_id) REFERENCES beatmap(beatmap_id)
);

```

NAPOMENA: polje `mods_used` je ključan dio svake ritmičke videoigre i pohranjuje informaciju o tome koje je modifikatore igrač koristio tijekom odigrane partije. Modifikatori su postavke koje mijenjaju način igranja mape, u našem slučaju kasnije koristimo 'DT' koji ubrzava pjesmu za 50%.

```

CREATE TABLE replay (
    replay_id      SERIAL          PRIMARY KEY,
    replay_data    BYTEA,
    upload_date    TIMESTAMP        NOT NULL,
    score_id       INTEGER          NOT NULL UNIQUE,
    CONSTRAINT fk_replay_score FOREIGN KEY (score_id) REFERENCES score(score_id)
);

```


4. Unos Podataka

Nakon kreiranja strukture baze, bilo je potrebno popuniti je inicijalnim podacima kako bi se omogućilo testiranje i demonstracija upita.

Iako je neke podatke bilo moguće za izmisliti, neki unosi došli su od stvarnih podataka iz već postojeće igre radi jednostavnosti.

Primjer unosa podataka za korisnike, profile, mape i rezultate:

```
INSERT INTO "user" (username, email, password, date_joined) VALUES
('-Milky', 'milky.game@example.com', 'hashed_milky_password', '2016-06-15 10:20:25'),
('Quiligru', 'quiligru.game@example.com', 'hashed_quiligru_password', '2017-01-20 10:41:05'),
('qxy', 'qxy.game@example.com', 'hashed_qxy_password', '2017-10-16 18:07:24'),
('Kuki', 'kuki.game@example.com', 'hashed_kuki_password', '2015-03-17 21:44:19'),
('Reform', 'reform.game@example.com', 'hashed_reform_password', '2013-12-11 18:50:52');

INSERT INTO profile (user_id, country, total_score, global_rank, country_rank, accuracy, level, total_pp) VALUES
((SELECT user_id FROM "user" WHERE username = '-Milky'), 'Croatia', 154889099268, 4284, 11, 98.92, 101, 10639),
((SELECT user_id FROM "user" WHERE username = 'Quiligru'), 'Bosnia and Herzegovina', 246615714160, 2419, 2, 98.50, 102, 11765),
((SELECT user_id FROM "user" WHERE username = 'qxy'), 'Croatia', 156178753560, 6301, 14, 98.62, 100, 9941),
((SELECT user_id FROM "user" WHERE username = 'Kuki'), 'Poland', 77975390415, 30057, 1320, 98.99, 100, 7051),
((SELECT user_id FROM "user" WHERE username = 'Reform'), 'Ireland', 225715238287, 3096, 13, 98.68, 102, 11289);

INSERT INTO beatmap (title, artist, max_possible_combo, bpm, length, difficulty_name, difficulty_rating, user_id) VALUES
('Bye Bye YESTERDAY', '3 Nen E Gumi Utatan', 557, 180, 90, 'Good Good Time', 5.43, 4),
('You Suck At Love (Speed Up Ver.)', 'Simple Plan', 428, 178, 77, 'Extra', 5.69, 5),
('Take You Down', 'Fox Stevenson', 432, 175, 59, 'Ultra', 5.25, 5),
('Desire (Cut Ver.)', 'Sub Focus, Dimension', 493, 174, 88, 'Expert', 4.95, 5),
('Poison (Cut Ver.)', 'Groove Coverage', 100, 178, 100, 'Chaos Elixir', 5.44, 4),
('Horrible Kids', 'Set It Off', 315, 170, 62, 'Cataclysm', 5.43, 4);

-- Rezultat: -Milky na 'Take You Down'
INSERT INTO score (score, accuracy, max_combo, miss_count, achieved_pp, date_played, mods_used, user_id, beatmap_id)
VALUES (1119453, 100.00, 432, 0, 545, '2022-11-19 20:55:20', 'HDDT', (SELECT user_id FROM "user" WHERE username = '-Milky'),
(SELECT beatmap_id FROM beatmap WHERE title = 'Take You Down'));

-- Rezultat: Quiligru na 'Take You Down'
INSERT INTO score (score, accuracy, max_combo, miss_count, achieved_pp, date_played, mods_used, user_id, beatmap_id)
VALUES (969672, 96.42, 402, 2, 348, '2022-11-19 23:54:30', 'HDDT', (SELECT user_id FROM "user" WHERE username = 'Quiligru'),
(SELECT beatmap_id FROM beatmap WHERE title = 'Take You Down'));

-- Rezultat: qxy na 'Take You Down'
INSERT INTO score (score, accuracy, max_combo, miss_count, achieved_pp, date_played, mods_used, user_id, beatmap_id)
VALUES (1027542, 97.13, 432, 0, 429, '2025-06-09 22:29:17', 'HDDT', (SELECT user_id FROM "user" WHERE username = 'qxy'),
(SELECT beatmap_id FROM beatmap WHERE title = 'Take You Down'));
```

NAPOMENA: Radi čitljivosti, inserte pripadnih rezultata sam unosio zasebno. Također, dva usera mi služe samo kao placeholderi za kreatore mapa koje sam insertao.

5. Jednostavni upiti

Prvi jednostavan upit

Ispis svih korisnika sortiranih prema datumu pridruživanja od najnovijih.

```
SELECT username, email, date_joined FROM "user" ORDER BY date_joined DESC;
```

	username character varying (50) 🔒	email character varying (100) 🔒	date_joined timestamp without time zone 🔒
1	qxy	qxy.game@example.com	2017-10-16 18:07:24
2	Quiligru	quiligru.game@example.com	2017-01-20 10:41:05
3	-Milky	milky.game@example.com	2016-06-15 10:20:25
4	Kuki	kuki.game@example.com	2015-03-17 21:44:19
5	Reform	reform.game@example.com	2013-12-11 18:50:52

Drugi jednostavan upit

Ovaj upit izlistava sve mape čiji je tempo (BPM) veći od 175.

```
SELECT title, artist, bpm FROM beatmap WHERE bpm > 175;
```

	title character varying (200) 🔒	artist character varying (200) 🔒	bpm integer 🔒
1	Bye Bye YESTERDAY	3 Nen E Gumi Utatan	180
2	You Suck At Love (Speed Up Ver.)	Simple Plan	178
3	Poison (Cut Ver.)	Groove Coverage	178

Treći jednostavan upit

Prikaz svih rezultata koji su odigrani sa 100% preciznosti.

```
SELECT score_id, score, accuracy FROM score WHERE accuracy = 100;
```

	score_id [PK] integer	score bigint	accuracy numeric (5,2)
1	6	1119453	100.00

Četvrti jednostavan upit

Prikaz svih korisnika koji su iz određene države, što je korisno za regionalne ljestvice.

```
SELECT * from profile WHERE country = 'Croatia';
```

	user_id [PK] integer	country character varying (50)	total_score bigint	global_rank integer	country_rank integer	accuracy numeric (5,2)	level smallint	total_pp integer
1	1	Croatia	154889099268	4284	11	98.92	101	10639
2	3	Croatia	156178753560	6301	14	98.62	100	9941

Peti jednostavan upit

Sljedeći upit izlistava samo one mape čija je ocjena težine veća od 5.5.

```
SELECT title, artist, difficulty_rating FROM beatmap WHERE difficulty_rating > 5.5;
```

	title character varying (200)	artist character varying (200)	difficulty_rating numeric (4,2)
1	You Suck At Love (Speed Up Ver.)	Simple Plan	5.69

6. Upiti nad više tablica

Prvi složeni upit

Ovaj upit generira 10 najboljih rezultata s imenima igrača i mapa (s obzirom na score).

```
SELECT u.username, b.title, s.score, s.accuracy
FROM score s
INNER JOIN "user" u USING(user_id)
INNER JOIN beatmap b USING(beatmap_id)
ORDER BY s.score DESC
LIMIT 10;
```

	username character varying (50) 	title character varying (200) 	score bigint 	accuracy numeric (5,2) 
1	-Milky	Bye Bye YESTERDAY	1145555	99.41
2	-Milky	Desire (Cut Ver.)	1138807	99.83
3	-Milky	Take You Down	1119453	100.00
4	Quiligrü	Desire (Cut Ver.)	1112023	99.78
5	-Milky	Horrible Kids	1081632	98.66
6	Quiligrü	Horrible Kids	1066772	99.24
7	qxy	Poison (Cut Ver.)	1037411	97.47
8	qxy	Take You Down	1027542	97.13
9	qxy	Horrible Kids	994625	98.00
10	-Milky	You Suck At Love (Speed Up Ver.)	983046	97.25

Drugi složeni upit

Prikaz svih igrača čiji rezultati imaju full combo, odnosno postigli su najveći mogući combo na pripadnoj mapi.

```
SELECT username, score, accuracy, achieved_pp
FROM "user"
INNER JOIN score s USING(user_id)
INNER JOIN beatmap b USING(beatmap_id)
WHERE s.max_combo = b.max_possible_combo
ORDER BY achieved_pp DESC;
```

	username character varying (50)	score bigint	accuracy numeric (5,2)	achieved_pp integer
1	-Milky	1145555	99.41	600
2	-Milky	1119453	100.00	545
3	qxy	994625	98.00	477
4	-Milky	1138807	99.83	452
5	Quiligru	1112023	99.78	446
6	qxy	1027542	97.13	429

Treći složeni upit

Ispis svih prijateljstva i njihov status zajedno s datumom početka prijateljstva.

```
SELECT u1.username AS player1, u2.username AS player2, f.status, f.friends_since
FROM friends f
INNER JOIN "user" u1 ON f.user_id_1 = u1.user_id
INNER JOIN "user" u2 ON f.user_id_2 = u2.user_id;
```

	player1 character varying (50)	player2 character varying (50)	status character varying (20)	friends_since date
1	-Milky	qxy	Accepted	2018-05-20
2	-Milky	Quiligru	Accepted	2019-11-11
3	Kuki	Reform	Pending	[null]

Četvrti složeni upit

Ispis svih mapa i imena njihovih kreatora.

```
SELECT b.title, b.artist, b.difficulty_name, u.username FROM beatmap b
INNER JOIN "user" u USING(user_id);
```

	title character varying (200) 🔒	artist character varying (200) 🔒	difficulty_name character varying (100) 🔒	username character varying (50) 🔒
1	Bye Bye YESTERDAY	3 Nen E Gumi Utatan	Good Good Time	Kuki
2	You Suck At Love (Speed Up Ver.)	Simple Plan	Extra	Reform
3	Take You Down	Fox Stevenson	Ultra	Reform
4	Desire (Cut Ver.)	Sub Focus, Dimension	Expert	Reform
5	Poison (Cut Ver.)	Groove Coverage	Chaos Elixir	Kuki
6	Horrible Kids	Set It Off	Cataclysm	Kuki

Peti složeni upit

Ispis svih korisnika koji za svoj rezultat imaju spremljen replay i na kojoj mapi.

```
SELECT u.username, s.score, b.title, r.upload_date
FROM score s
INNER JOIN replay r USING(score_id)
INNER JOIN "user" u USING(user_id)
INNER JOIN beatmap b USING(beatmap_id)
```

	username character varying (50) 🔒	score bigint 🔒	title character varying (200) 🔒	upload_date timestamp without time zone 🔒
1	-Milky	1145555	Bye Bye YESTERDAY	2025-06-15 11:10:21.666793

7. Upiti s agregirajućim funkcijama

Prvi upit

Ispis prosječne preciznosti za svakog korisnika (na temelju unesenih rezultata – scores)

```
SELECT u.username, ROUND(AVG(s.accuracy), 2) AS average_accuracy
FROM score s
INNER JOIN "user" u USING(user_id)
GROUP BY u.username
ORDER BY average_accuracy DESC;
```

	username character varying (50) 🔒	average_accuracy numeric 🔒
1	-Milky	99.03
2	Quiligru	98.39
3	qxy	94.93

Drugi upit

Upit prebrojava koliko igrača ima iz pojedine države.

```
SELECT country, count(user_id) AS number_of_players
FROM profile
GROUP BY country
ORDER BY number_of_players DESC;
```

	country character varying (50) 🔒	number_of_players bigint 🔒
1	Croatia	2
2	Ireland	1
3	Bosnia and Herzegovina	1
4	Poland	1

Treći upit

Upit pronalazi najveće ostvaren pp (performance points) za svaku mapu.

```
SELECT b.title, MAX(s.achieved_pp) AS max_pp_on_map
FROM beatmap b
INNER JOIN score s USING(beatmap_id)
GROUP BY b.title
ORDER BY max_pp_on_map DESC;
```

	title character varying (200) 🔒	max_pp_on_map integer 🔒
1	Bye Bye YESTERDAY	600
2	You Suck At Love (Speed Up Ver.)	563
3	Take You Down	545
4	Horrible Kids	509
5	Poison (Cut Ver.)	505
6	Desire (Cut Ver.)	452

Četvrti upit

Upit prikazuje ukupan broj promašaja za svakog korisnika.

```
SELECT u.username, SUM(miss_count) AS total_misses
FROM score s
INNER JOIN "user" u USING(user_id)
GROUP BY u.username
ORDER BY total_misses ASC;
```

	username character varying (50) 🔒	total_misses bigint 🔒
1	Quiligrü	4
2	-Milky	4
3	qxy	21

Peti upit

Izbroji ukupan broj rezultata koji je svaki korisnik odigrao.

```
SELECT u.username, COUNT(s.score_id) AS total_plays
FROM "user" u
JOIN score s USING(user_id)
GROUP BY u.username
ORDER BY total_plays DESC;
```

	username character varying (50) 🔒	total_plays bigint 🔒
1	qxy	6
2	-Milky	6
3	Quiligru	4

8. Podupiti / ugniježdeni upiti / skupovne operacije

Prvi upit

Prikaži korisnike koji imaju više total_pp od prosjeka svih korisnika.

```
SELECT username, total_pp
FROM "user" INNER JOIN profile USING(user_id)
WHERE total_pp > (SELECT AVG(total_pp) FROM profile)
```

	username character varying (50) 🔒	total_pp integer 🔒
1	-Milky	10639
2	Quiligru	11765
3	Reform	11289

Drugi upit

Prikaži sve mape na kojima je korisnik '-Milky' postavio rezultat.

```
SELECT title, artist
FROM beatmap
WHERE beatmap_id IN (SELECT beatmap_id FROM score
WHERE user_id = (SELECT user_id FROM "user" WHERE username = '-Milky'))
```

	title character varying (200) 🔒	artist character varying (200) 🔒
1	Horrible Kids	Set It Off
2	Bye Bye YESTERDAY	3 Nen E Gumi Utatan
3	Take You Down	Fox Stevenson
4	Poison (Cut Ver.)	Groove Coverage
5	Desire (Cut Ver.)	Sub Focus, Dimension
6	You Suck At Love (Speed Up Ver.)	Simple Plan

Treći upit

Pronađi korisnike koji još nisu postavili nijedan rezultat.

```
SELECT username
FROM "user" u
WHERE NOT EXISTS (SELECT 1 FROM score s WHERE s.user_id = u.user_id);
```

	username character varying (50) 🔒
1	Kuki
2	Reform

Četvrti upit

Prikaži sve korisnike iz hrvatske i poljske koristeći skupovnu operaciju UNION.

```
SELECT username, country FROM "user" INNER JOIN profile USING(user_id) WHERE country = 'Croatia'
UNION
SELECT username, country FROM "user" INNER JOIN profile USING(user_id) WHERE country = 'Poland';
```

	username character varying (50) 🔒	country character varying (50) 🔒
1	-Milky	Croatia
2	Kuki	Poland
3	qxy	Croatia

Peti upit

Prikaži rezultat s najvećim ostvarenim pp od svih rezultata.

```
SELECT u.username, b.title, s.achieved_pp
FROM score s
INNER JOIN "user" u USING(user_id)
INNER JOIN beatmap b USING(beatmap_id)
WHERE s.achieved_pp = (SELECT MAX(achieved_pp) FROM score);
```

	username character varying (50) 🔒	title character varying (200) 🔒	achieved_pp integer 🔒
1	-Milky	Bye Bye YESTERDAY	600

9. Zadane vrijednost

Prva zadana vrijednost:

Dodajemo default vrijednost na stupac level što pojednostavljuje unos jer se te vrijednosti onda ne moraju eksplicitno navoditi.

```
ALTER TABLE profile ALTER COLUMN level SET DEFAULT 1;
```

Druga zadana vrijednost:

Analogno, ukoliko igrač nije zabilježio niti jedan rezultat, default vrijednost ukupnog score trebala bi biti 0.

```
ALTER TABLE profile ALTER COLUMN total_score SET DEFAULT 0;
```

Treća zadana vrijednost:

Ukoliko je poslan zahtjev za prijateljstvo status bi trebao biti pending.

```
ALTER TABLE friends ALTER COLUMN status SET DEFAULT 'Pending';
```

10. Uvjeti

Uvodimo CHECK uvjete kako bi osigurali integritet podataka.

Prvi uvjet:

Preciznost bi uvijek trebala biti unutar [0, 100], stoga ovaj uvjet osigurava da preciznost ne može biti izvan logičkog raspona.

```
ALTER TABLE profile ADD CONSTRAINT check_accuracy CHECK (accuracy >= 0 AND accuracy <= 100);
```

Drugi uvjet:

Tempo, odnosno BPM bi uvijek trebao biti veći od 0.

```
ALTER TABLE beatmap ADD CONSTRAINT check_bpm CHECK(bpm > 0);
```

Treći uvjet:

Ovaj uvjet sprječava duple unose i mogućnost korisnika da bude prijatelj sam sa sobom.

```
ALTER TABLE friends ADD CONSTRAINT check_friends_order CHECK (user_id_1 < user_id_2);
```

11. Komentari

Koristimo naredbu COMMENT ON kako bi se dokumentirala svrha tablica i stupaca direktno unutar baze.

```
COMMENT ON TABLE "user" IS 'Pohranjuje osnovne podatke za prijavu i identifikaciju korisnika.';
COMMENT ON COLUMN beatmap.user_id IS 'ID korisnika koji je kreirao mapu.';
COMMENT ON COLUMN score.achieved_pp IS 'Performance points -> mjera vještine za određeni score.';
```

12. Dodavanje indeksa

Kreirani su standardni B-tree indeksi na stupcima koji se najčešće pretražuju kako bi se ubrzao dohvat podataka.

```
CREATE INDEX user_username ON "user"(username);
CREATE INDEX score_user_beatmap ON score(user_id, beatmap_id);
CREATE INDEX friends_status ON friends(status);
```

13. Procedure

Prva procedura

Ova procedura automatizira proces registracije. Ona prima podatke o novom korisniku, unosi ih u tablicu user, dohvaća novostvoreni user_id i odmah kreira povezani redak u tablici profile.

```
CREATE OR REPLACE PROCEDURE add_new_user(  
    p_username VARCHAR,  
    p_email VARCHAR,  
    p_password VARCHAR,  
    p_country VARCHAR  
)  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    new_user_id INTEGER;  
BEGIN  
    INSERT INTO "user"(username, email, password, date_joined)  
    VALUES(p_username, p_email, p_password, NOW())  
    RETURNING user_id INTO new_user_id;  
  
    INSERT INTO profile(user_id, country)  
    VALUES(new_user_id, p_country);  
END;  
$$;
```

Druga procedura

Ključna procedura za društvenu komponentu igre. Ona omogućuje jednom korisniku da prihvati zahtjev za prijateljstvo od drugog.

Procedura provjerava postoji li zahtjev na čekanju (Pending), ažurira mu status u Accepted i postavlja trenutni datum kao početak prijateljstva.

```
CREATE OR REPLACE PROCEDURE accept_friend_request(
    p_requesting_user_id INTEGER,
    p_accepting_user_id INTEGER
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_user1 INTEGER;
    v_user2 INTEGER;
    request_exists INTEGER;
BEGIN
    -- osigurava da je manji ID uvijek v_user1
    IF p_requesting_user_id < p_accepting_user_id THEN
        v_user1 := p_requesting_user_id;
        v_user2 := p_accepting_user_id;
    ELSE
        v_user1 := p_accepting_user_id;
        v_user2 := p_requesting_user_id;
    END IF;

    SELECT COUNT(*) INTO request_exists
    FROM friends
    WHERE user_id_1 = v_user1
        AND user_id_2 = v_user2
        AND status = 'Pending';

    IF request_exists = 1 THEN
        UPDATE friends
        SET status = 'Accepted',
            friends_since = CURRENT_DATE
        WHERE user_id_1 = v_user1
            AND user_id_2 = v_user2;
    ELSE
        RAISE EXCEPTION
        'Nije pronađen zahtjev za prijateljstvo
        na čekanju između korisnika % and %.', p_requesting_user_id, p_accepting_user_id;
    END IF;
END;
$$;
```


Ako napravimo sljedeći upit dobit ćemo iduće:

```
SELECT * FROM friends WHERE friends.user_id_1 = 4;
```

	user_id_1 [PK] integer	user_id_2 [PK] integer	friends_since date	status character varying (20)
1	4	5	[null]	Pending

Nakon poziva procedure sada upitom dobivamo:

```
CALL accept_friend_request(4,5);
```

	user_id_1 [PK] integer	user_id_2 [PK] integer	friends_since date	status character varying (20)
1	4	5	2025-06-15	Accepted

Ponovnim pozivom procedure dobivamo:

```
ERROR: Nije pronađen zahtjev za prijateljstvo  
       na čekanju između korisnika 4 and 5.  
CONTEXT: PL/pgSQL function accept_friend_request(integer,integer) line 32 at RAISE  
  
SQL state: P0001
```

14. Okidači

Prvi okidač

Ovaj okidač se aktivira prije inserta na tablicu score. Njegova svrha je provjeriti sadrži li ubačeni rezultat modifikator ‘DT’ (Double Time). Ako da, automatski množi osnovni BPM i težinu mape s 1.5 i sprema te nove efektivne vrijednosti.

```
CREATE OR REPLACE FUNCTION trig_calc_effective_stats_func()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    base_bpm INTEGER;
    base_difficulty NUMERIC;
BEGIN
    SELECT bpm, difficulty_rating INTO base_bpm, base_difficulty
    FROM beatmap
    WHERE beatmap_id = NEW.beatmap_id;

    IF NEW.mods_used ILIKE '%dt%' THEN
        NEW.effective_bpm := base_bpm * 1.5;
        NEW.effective_difficulty_rating := base_difficulty * 1.5;
    ELSE
        NEW.effective_bpm := base_bpm;
        NEW.effective_difficulty_rating := base_difficulty;
    END IF;

    RETURN NEW;
END;
$$

CREATE TRIGGER trig_effective_score
BEFORE INSERT ON score
FOR EACH ROW
EXECUTE FUNCTION trig_calc_effective_stats_func();
```

NAPOMENA: iako se modifikator zove ‘Double Time’, po konvenciji je to najčešće 1.5, a ne 2.

Prije kreiranja okidača sljedeći upit bi davao:

```
SELECT score_id, score, effective_bpm, effective_difficulty_rating FROM score
ORDER BY score DESC
LIMIT 5;
```

	score_id [PK] integer	score bigint	effective_bpm integer	effective_difficulty_rating numeric (4,2)
1	1	1145555	[null]	[null]
2	14	1138807	[null]	[null]
3	6	1119453	[null]	[null]
4	16	1112023	[null]	[null]
5	12	1081632	[null]	[null]

Nakon kreiranja okidača i ponovnog unosa rezultata upitom dobivamo:

	score_id [PK] integer	score bigint	effective_bpm integer	effective_difficulty_rating numeric (4,2)
1	1	1145555	270	8.15
2	14	1138807	261	7.43
3	6	1119453	263	7.88
4	16	1112023	261	7.43
5	12	1081632	255	8.15

Drugi okidač

Ovaj okidač se također aktivira prije inserta na tablicu score. On sprječava unos novog rezultata ukoliko on nije bolji od korisnikovog osobnog rekorda na toj istoj mapi. Ovim načinom se sprječava pretrpavanje baze nepotrebnim podacima, a istovremeno osigurava da se bilježi samo napredak igrača.

```
CREATE OR REPLACE FUNCTION check_score_improvement()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    highest_score BIGINT;
BEGIN
    SELECT MAX(score) INTO highest_score
    FROM score
    WHERE user_id = NEW.user_id AND beatmap_id = NEW.beatmap_id;

    IF highest_score IS NOT NULL AND NEW.score <= highest_score THEN
        RAISE EXCEPTION 'Novi rezultat (score: %) nije bolji od postojećeg
        najboljeg rezultata (score: %). Unos je otkazan.', NEW.score, highest_score;
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_before_score_insert_improvement
BEFORE INSERT ON score
FOR EACH ROW
EXECUTE FUNCTION check_score_improvement();
```

Sada prilikom inserta rezultata koji ima manju vrijednost od već postojećeg dobivamo:

```
ERROR:  Novi rezultat (score: 130000) nije bolji od postojećeg
        najboljeg rezultata (score: 1112023). Unos je otkazan.
CONTEXT:  PL/pgSQL function check_score_improvement() line 10 at RAISE

SQL state: P0001
```

15. Zaključak

Iako ovaj model pokriva sve osnovne funkcionalnosti, prostor za nadogradnju je velik. Budući koraci mogli bi uključivati implementaciju sustava za turnire, detaljniju statistiku po modifikatorima ili uvođenje tablica za korisničke vizualne dodatke (skinove). Sve u svemu, ovaj projekt predstavlja čvrst temelj na kojem bi se mogla graditi kompletna i uspješna online ritmička igra.